

Reading RTL for Assets

What a hardware security asset actually is, how to spot one in VHDL, and why *control* information is the part a parser cannot afford to lose.

01 What an asset is	02 C, I, A and U	03 Hardware is different
04 Trust boundaries	05 Value flow vs control flow	06 Six patterns in RTL
07 Thinking like an attacker	08 What the parser owes	09 Judging a parse yourself

01

What an asset is

And the mistake almost everyone makes first

An asset is something worth protecting. In hardware, it is a specific element — a register, a port, a signal — where **an attacker doing something to it causes a security problem**.

Three verbs matter. An attacker might:

- **read** it, and learn something they should not know;
- **change** it, and make the system behave in a way it should not;
- **block** it, and stop the system doing something it must.

THE COMMON MISTAKE

Importance is not the test. A clock is essential to every register in a design. Remove it and nothing works. It is almost never an asset.

Why? Because an attacker who can stop your clock already owns the chip. The clock is not the thing they are attacking. It is not a step on a path — it is the floor.

The correct test has two halves, and both must hold:

1. Is there a **plausible way** for an attacker to reach it?
2. Does reaching it produce a **security** consequence, not just a functional one?

A signal that fails the first is unreachable. A signal that fails the second is just a bug.

Assets are usually about authority

Here is the shortest useful heuristic.

REMEMBER THIS

Data-carrying signals are rarely assets. **Authority-carrying signals usually are.**

A wire that moves a value from A to B is plumbing. A bit that decides *whether* the move is allowed is a control point — and control points are what attackers go after.

This one idea explains most of the rest of this document.

02

C, I, A and U

Four questions to ask about any candidate

Once you suspect an element is an asset, you have to say *what kind*. Four categories cover it. Your project uses all four.

C Confidentiality

The value must not be readable by the wrong party.

Examples: a key register, the output of a random number generator, a buffer holding decrypted data. The question: *if someone reads this, what do they learn?*

I Integrity

The value must not be changeable by the wrong party.

Examples: a memory-permission register, a boot configuration, a lock bit, a counter used for replay protection. The question: *if someone writes this, what stops being true?*

A Availability

The function must not be blockable.

Examples: a watchdog timer, a bus arbiter, a hardware lock. The question: *if someone jams this, what stops working — and does that matter for security, or only for uptime?*

U Unauthorised access

In your project, U is reserved for real **override, bypass or injection** paths. It is the narrowest of the four, on purpose.

Examples: a debug interface that can write CPU state, a test mode that skips a check, a backdoor register. The question: *does this let someone step around a control instead of defeating it?*

KEEP U NARROW

U is the category that inflates fastest. Almost anything can be described as "could lead to unauthorised access" if you squint. Reserve it for a path that genuinely *skips* a check — not one that merely influences a value which later feeds a check.

One element can carry more than one. The PMP configuration register in your design is an **Integrity** asset first — if you can write it, permissions are gone. It is arguably a U asset too, since changing it bypasses the check rather than breaking it.

03

Hardware is different from software

Four differences that change what counts

Software security intuitions transfer badly. Four reasons.

1. You cannot patch it

A software bug gets an update. A silicon bug is in every chip ever shipped. This raises the cost of a missed asset enormously, and it is why identification happens before tape-out, on RTL, rather than after.

2. The attacker may hold the chip

Physical access is a normal assumption, not an extreme one. That makes debug ports, test modes and scan chains first-class attack surfaces. In software these have no equivalent.

3. Everything is parallel and always on

In software, a function that is not called does nothing. In hardware, every gate is live every cycle. A path that "should never be taken" is still physically there and still driven. Dead code is not dead.

4. Trust boundaries are physical

In software a boundary is a check in a function. In hardware it is a specific wire crossing between two blocks, and you can point at it in the source. That is good news: **boundaries are findable by reading RTL.**

CONSEQUENCE

Because boundaries are physical, asset identification on RTL is a *structural* task before it is a judgement task. You find the boundaries first, then ask what crosses them.

04

Trust boundaries

Where assets live

A trust boundary separates two parties who do not fully trust each other. Assets cluster at boundaries, because that is where one side's authority is enforced against the other.

An SoC like NEORV32 has four you can find in the source.

Privilege levels

RISC-V has machine mode (M), supervisor mode (S) and user mode (U). M mode can do anything. U mode is restricted. The boundary between them is enforced by hardware.

In RTL, look for a signal carrying the current mode, and for comparisons against it.

```
-- the privilege of the current access, from your PMP
acc_priv -- one bit: is this access privileged?
allow(x) <= pmpcfg(x)(cfg_r_c) or (not pmpcfg(x)(cfg_l_c)); -- M-mode path
```

Anything that decides or carries the privilege level is an asset candidate.

Debug versus normal operation

Debug hardware exists to inspect and change state that normal code cannot touch. That is exactly what an attacker wants. The boundary is whatever enables debug.

In NEORV32, look at `neorv32_debug_dtm` — a transport module that can drive requests into the debug module.

Bus masters

When two masters share a bus, one may be trusted and the other not — a CPU and a DMA engine, for instance. The arbiter decides who wins, and any lock or priority bit is a control point.

Your `neorv32_bus` module is full of these. So is `neorv32_hwspinlock`.

The chip boundary

Anything reaching a pin is observable, and often drivable, from outside. An output port that carries internal state is a confidentiality question. An input port that reaches a control path is an integrity question.

HOW TO USE THIS

Before reading any single element, list the boundaries in the module. Then ask of each element: **does it cross one, enforce one, or configure one?** Elements that do none of those three are rarely assets.

Value flow versus control flow

The single most useful distinction in this whole document

Every coupling between two signals is one of two kinds.

Value flow – "A becomes part of B"

```
b <= a;           -- a flows into b
c <= a + d;       -- a contributes to c
```

You can see this by reading one line. Everything you need is in the assignment.

Control flow – "A decides whether B happens"

```
if (en = '1') then
  b <= a;           -- en decides. But en is nowhere in b's value.
end if;
```

You cannot see this by reading the assignment. `b <= a` says nothing about `en`. You have to look at *the branch you are standing inside*.

WHY THIS MATTERS MORE THAN ANYTHING ELSE

Security lives in control flow. "Who is allowed to write this" is a control question. "What value gets written" is a data question.

And control flow is exactly the kind that is invisible when you read an assignment on its own.

A worked example

From your PMP module. Two lines, two very different kinds of information.

```
-- VALUE FLOW: easy to see, easy to record
cmp_ge(r) <= '1' when TOR_EN else '0';
    cmp_ge derives from TOR_EN. Visible on the line.

-- CONTROL FLOW: invisible unless you look at the enclosing branch
if (pmpcfg_we(i/4) = '1') and (pmpcfg(i)(cfg_l_c) = '0') then
  pmpcfg(i) <= ...
    The lock bit decides whether config can be written at all.
    Nothing in pmpcfg's assigned VALUE mentions the lock bit.
```

The second one is the security mechanism of the entire module. The first is arithmetic.

If a tool records only what it can see in assignments, it will capture the first perfectly and lose the second. **That is precisely what your parser does** — 83% of its edges are value flow, 9% are control.

Six patterns you can recognise in RTL

Shapes that reliably indicate an asset

These are visual patterns. Once you know them you spot them by shape, before you read the signal names.

I The lock bit A bit that, once set, prevents writes to something else. Usually stays set until reset. <pre>if (we='1') and (cfg(L)='0') then cfg <= wdata;</pre>	I The permission check A comparison whose result allows or refuses an operation. <pre>allow <= cfg(R) or (not cfg(L)); fault <= req and (not allow);</pre>
U The mode gate A branch taken only in a privileged or debug mode. The mode signal is the asset. <pre>if (priv = M_MODE) then -- unrestricted path</pre>	C The entropy source A value that is generated, not computed. Cannot be recomputed from inputs. <pre>data_o <= sample_reg; -- driven by a free-running source</pre>
A The arbiter Decides which of several requesters proceeds. Jamming it starves the others. <pre>if (locked = '1') then grant <= keep_current;</pre>	U The override path An alternative source that replaces the normal one. Test and debug hooks look like this. <pre>if (dbg_en = '1') then pc <= dbg_addr; -- bypass</pre>

Notice what five of the six have in common. The security-relevant part is inside an `if` condition, not in the assigned value. That is why control flow is not a detail — it is the substance.

07

Thinking like an attacker

Turning a suspicion into a real finding

Finding a control point is not enough. You have to show a path.

Ask three questions in order. If any answer is no, it is probably not an asset.

1. Can anything outside this block reach it?

Trace backwards. Is it driven by a port? Does the port come from a bus that software can write? Is it configured by a register that user-mode code can touch?

An internal signal that only ever gets values from other internal signals is usually out of reach.

2. What is the smallest change that hurts?

Be specific. Not "corrupting this would be bad" but "clearing this one bit re-opens writes to a locked region".

If you cannot name the bit and the effect, you have not found the mechanism yet.

3. Does the effect cross a boundary?

An attacker who already has machine mode gains nothing from writing a machine-mode register. The change has to move authority from one side of a boundary to the other.

THE TEST IN ONE SENTENCE

Name the attacker, the reach, the bit, and the boundary crossed. If you can fill in all four, it is an asset. If any slot is empty, keep reading the RTL.

08

What the parser owes the asset stage

And why some things cannot be recovered later

Your pipeline has two stages. The parser describes elements. The asset stage judges them.

The asset stage never sees the RTL. It sees only what the parser wrote. So anything the parser leaves out is gone for good.

Recoverable versus unrecoverable

Not all omissions are equal. Some the asset stage can work around. Some it cannot.

WHAT THE PARSER RECORDS	IF IT IS MISSING	RECOVERABLE?
function / functionality	Description is vague	Partly — other fields hint at it
kind (register vs signal)	Cannot tell state from wire	Partly — CAPTURES edges imply it
SOURCES / CARRIES edges	Data path unknown	Partly — often implied both ways
GATES / SELECTS edges	Control structure unknown	No
the condition itself	Cannot tell a lock from an enable	No

The last two rows are the whole problem.

A control coupling leaves no trace anywhere else. If the parser does not write `A GATES B`, then nothing in the record says A has any relationship to B at all. The asset stage cannot infer it, because there is nothing to infer from.

THE DESIGN PRINCIPLE THIS GIVES YOU

A parser stage should be **generous with control information and strict about judgement**.

Describing that one element governs another is a structural fact — it is safe, checkable, and irreplaceable. Deciding whether that matters is the next stage's job, and doing it early biases everything after.

Why "the prose said it" is not enough

Your parser often *describes* gating correctly in the role text and then fails to emit the edge. That looks like a near miss. It is a total miss.

Prose is not queryable. The asset stage cannot follow a chain of "gates" written in English across two hundred elements. Structured edges are what make reach traceable. A fact that exists only in a sentence is a fact the pipeline cannot use.

09

Judging a parse yourself

A checklist you can apply without any tooling

Pick one module you know well. Open the RTL beside the parse. Work through these.

☐ Every port and signal in the RTL has an entry. Nothing invented, nothing missing.

☐ Every element that appears in an

`if` ,

`case` or

`when` condition has at least one control edge — GATES, SELECTS, CONSTRAINS or OVERRIDES.

☐ For each control edge, the condition is recorded somewhere, not just the fact of gating.

☐ Registers are marked as registers. Combinational wires are not.

☐	Reset behaviour is distinguishable from clocked behaviour.
☐	Targets name real declared elements, so a chain can be followed.
☐	Every element that crosses the entity boundary says so, and says which direction.
☐	No annotation states or implies whether something is important, sensitive or secure.

The second and third boxes are the ones that fail most often, and they are the ones that matter most.

A quick numeric check

You can do this in a few lines of script, on any parse:

1. Count elements that appear in a branch condition in the RTL. Call it **N**.
2. Count how many of those have a control edge in the parse. Call it **M**.
3. $M \div N$ is your control coverage.

Your project already has this as `governing_coverage()`. It currently reads **51%**.

There is no universal threshold. But you can reason about it: if half the control structure is absent, then any asset the stage finds by following control paths is found by luck, and any it misses was never findable.

THE ONE THING TO TAKE AWAY

Assets are control points. Control points are visible only in conditions. Conditions are the hardest thing for a language model to notice, because they are not in the line it is reading.

So the quality of an RTL parser, for security purposes, is very close to its control coverage — and almost nothing else.

Where to go next

Reading that will deepen each part

- **The RISC-V privileged specification** — the actual definition of M/S/U mode and of PMP. Read the PMP chapter closely; it explains the lock bit properly.
- **Your own RTL** — `neorv32_cpu_pmp.vhd` is the best teaching example in the corpus. It contains four of the six patterns in section 06.
- **CWE hardware view (CWE-1194)** — a catalogue of hardware weakness classes. Useful as a checklist of what goes wrong, and organised by mechanism rather than by product.
- **Your ablation log** — the measurements in it are more specific to your pipeline than any general reference.

This document gives you the model. The Atlas gives you the measurements. Between them you should be able to write the next parser prompt yourself — and, more usefully, to tell whether it worked.